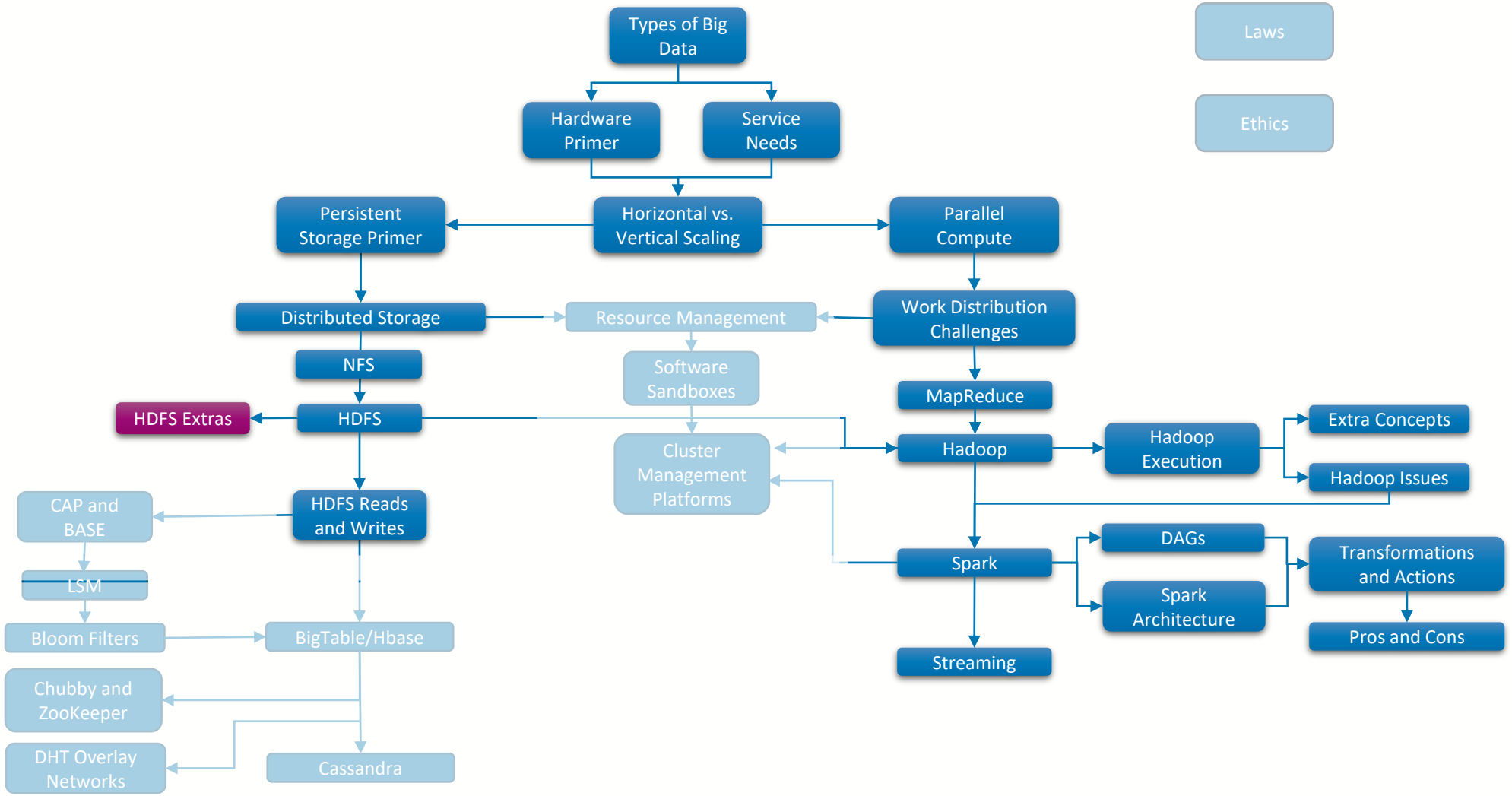
W6-01: HDFS States, Recovery and More

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

University of Glasgow

Where Are We?

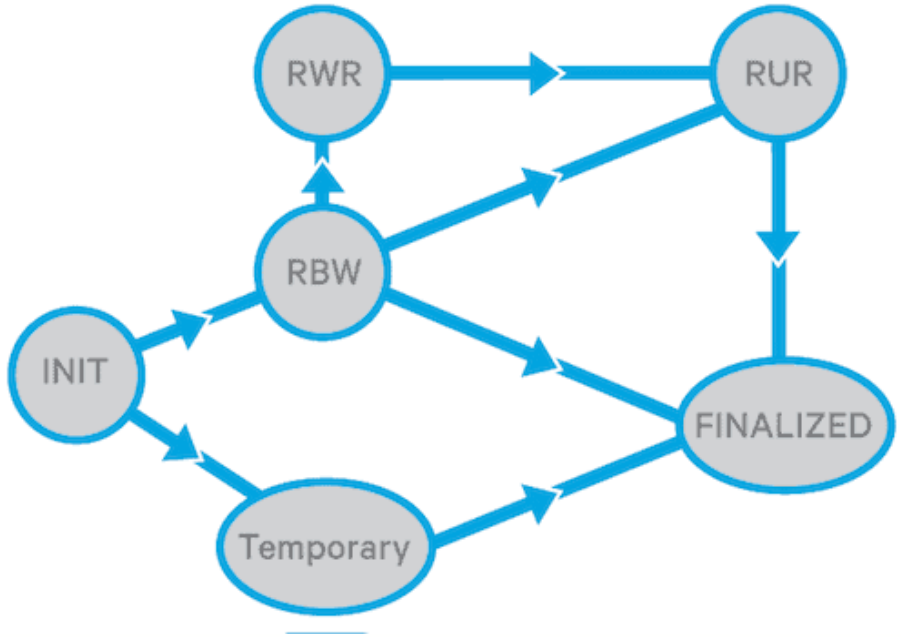


States and Leases

Write Failures and Block States

- During HDFS writes, individual Blocks can be in a range of defined **Data Node States**
  - Replica Being Written (RBW): Data is being written to the block (write or append) but is not yet complete
  - Replica Waiting to be Recovered (RWR): If a Data Node fails during a write all of its RBW blocks are marked as RWR.
  - Replica Under Recovery (RUR): If the Data Node comes back online quickly, then it will report its RWR blocks to the Name Node and they can be used during the recovery process, when being recovered they are marked as RUR
  - Finalized: Blocks are finalized when all data has been written in the block. Finalized blocks are given a generation stamp, all blocks with the same generation stamp should have the same data

  - (There is also a Temporary state, which is used during block replication)



Block Replica State Transitions

https://blog.cloudera.com/understanding-hdfs-recovery-processes-part-1/

University of Glasgow

Name Node Write View

- When a Client asks to write a Block (with a certain number of replicas), the Name Node sends the list of Data Nodes that it should write to (known as the pipeline)
  - The Name Node also tracks the write process via the heartbeats it received from those Data Nodes in the pipeline

- From the Name Node perspective, a Block can be in 5 states:
  - Under Construction: Data Node reports the Block as RBW
  - Under Recovery: If the block's lease expires during the write it will enter the under recovery state when block recovery begins
  - Committed: At least one replica of the Block has been finalised, but there are fewer than the required number of replicas
  - Complete: There are at least the required number of Block replicas in the finalized state

University of Glasgow

File Locks Vs. Leases

- The NFS we looked at earlier locked files when a user wanted to write
  - Locks last until the user releases it
    - https://docstore.mik.ua/orelly/networking_2ndEd/nfs/ch11_02.htm

- The HDFS provides a lease to the user when they want to write a file
  - This lease will expire when the file is closed by the writer or…
  - a period of time has elapsed (see the discussion on hard and soft limits in the paper)

- Why is it important to have a lease here?
  - We don't want to have a block in editable mode for extended periods of time, as we need to replicate changes
  - A client may fail in some way since its user code

University of Glasgow

Failure Recovery

University of Glasgow

Lease Manager and Lease Recovery

- Within the Name Node there is a Lease Manager
  - Tracks the write leases for each File

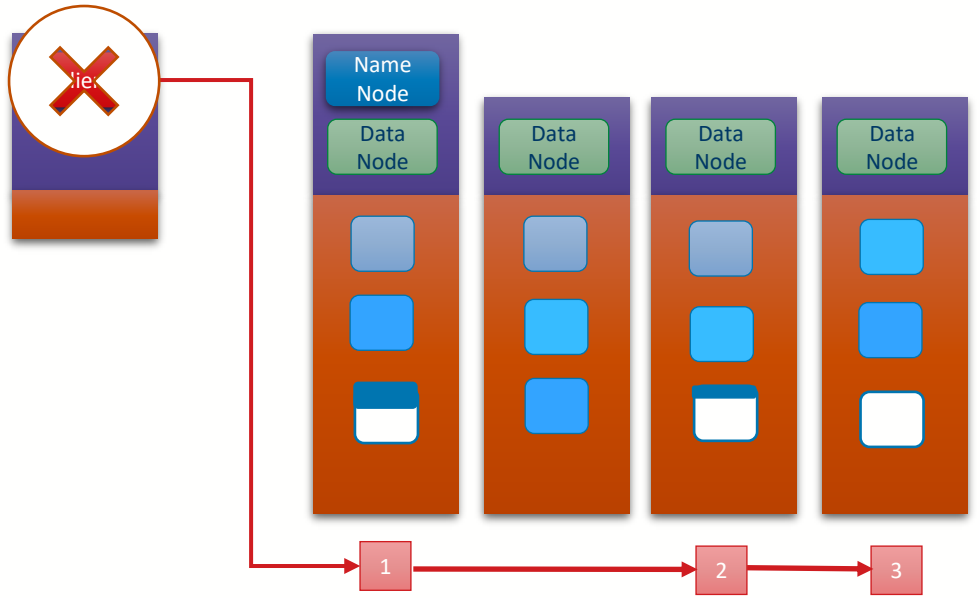
- If an HDFS lease is not explicitly renewed by the deadline (e.g. because the client holding it dies), then the Name Node will mark it as expired and trigger Lease Recovery

- Lease Recovery:
  - If no Write was taking place this will simply close the file
  - Otherwise, this will trigger Block Recovery

Block Recovery

- Lets assume a lease expires during a block write
  - Some data nodes will have partial data
  - We need to Finalize the block

- Assuming no Data Node failures, the pipeline will still be intact, this means that we will shortly reach data consistency amongst the data nodes

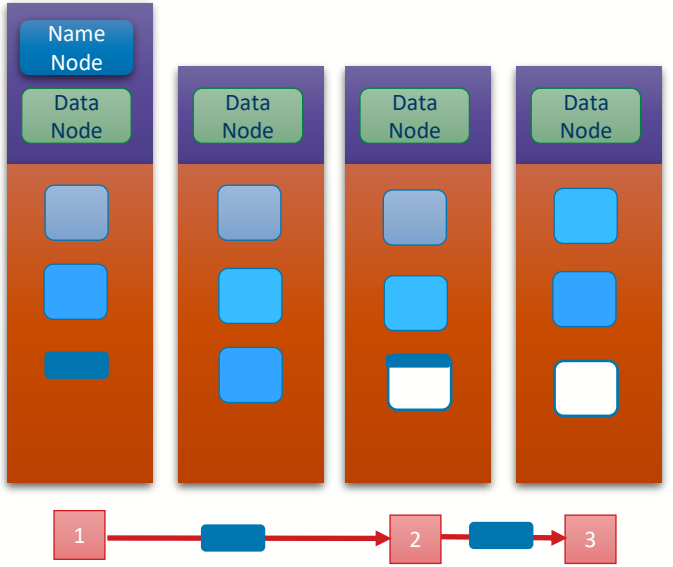


University of Glasgow

Block Recovery

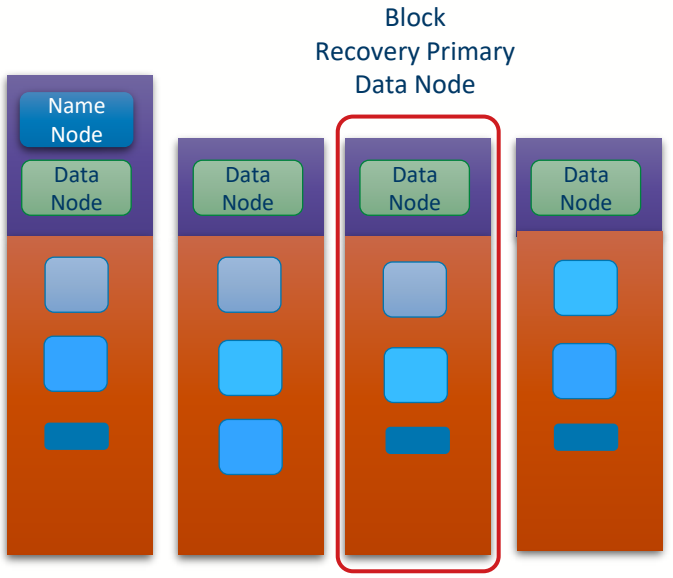
- Lets assume a lease expires during a block write
  - Some data nodes will have partial data
  - We need to Finalize the block

- Assuming no Data Node failures, the pipeline will still be intact, this means that we will shortly reach data consistency amongst the data nodes



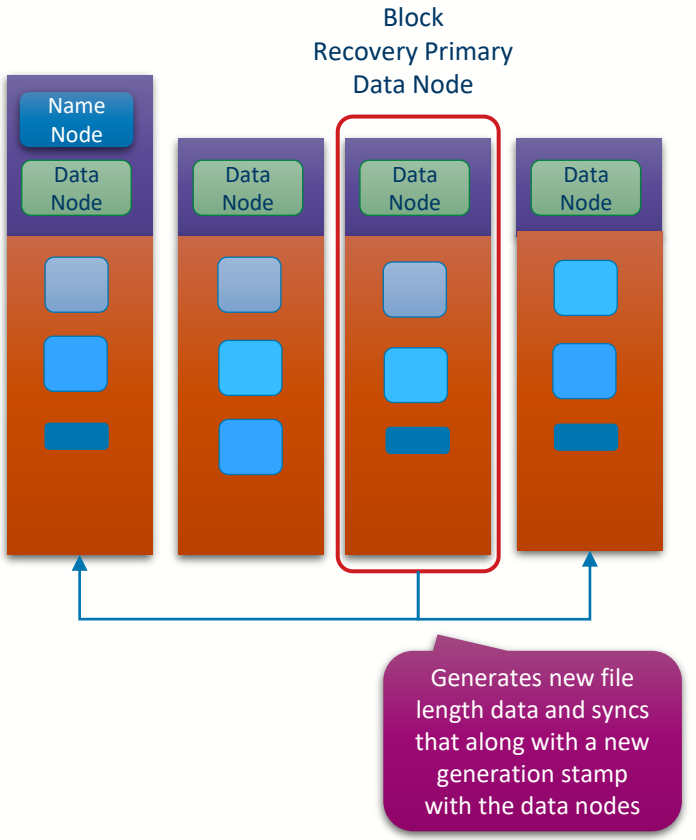
Block Recovery

- Lets assume a lease expires during a block write
  - Some data nodes will have partial data
  - We need to Finalize the block
- Assuming no Data Node failures, the pipeline will still be intact, this means that we will shortly reach data consistency amongst the data nodes
- Because the Client is now unavailable it cannot Finalize the block
  - Instead the Name Node will assign one of the Data nodes to perform this role instead
  - This is known as Block Recovery



Block Recovery

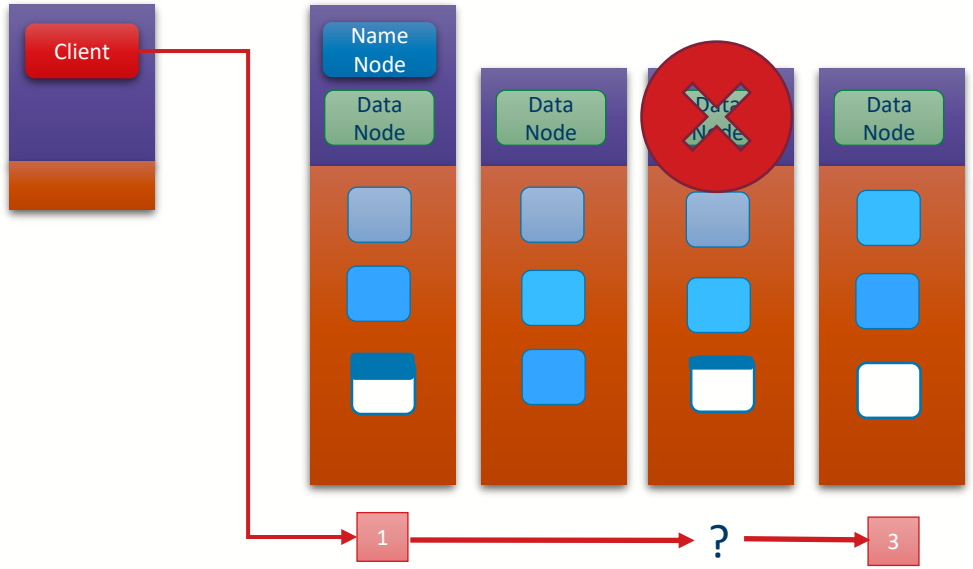
- Lets assume a lease expires during a block write
  - Some data nodes will have partial data
  - We need to Finalize the block
- Assuming no Data Node failures, the pipeline will still be intact, this means that we will shortly reach data consistency amongst the data nodes
- Because the Client is now unavailable it cannot Finalize the block
  - Instead the Name Node will assign one of the Data Nodes to perform this role instead
  - This is known as Block Recovery



Pipeline Recovery

- The other type of failure we care about is when a Data Node fails during the write process

- Three Scenarios
  - Setup Failure
    - Client abandons the write and tries the process from scratch
  - Failure During Write
    - If Failure is self detected by the Data Node, then it acts dead (cuts TCP-IP connection)
    - Client detects the missing Data Node and stops sending data, then reconstructs the pipeline using the remaining nodes
  - Close Failure
    - Client reconstructs the pipeline and re-tries



Additional Notes

University of Glasgow

Name Node and Data Node Communication

- **Data Nodes communicate with the Name Node, not the other way around**

- When a Data Node starts it performs a handshake with the Name Node
  - Checks that the namespace is correct
  - Then it registers, receiving a unique StorageID

- When a Data Node finishes receiving a Block, it sends a Block Report to the Client
  - List of (block_id, timestamp, length) for every block in the Data Node

- Data Nodes also send heartbeats to the Name Node
  - Periodic (every 3 seconds)
  - Provides information about storage use and capacity, as well as Block Reports
  - Used to detect Data Node failures

University of Glasgow

Re-Replication

- Responsibility of the Name Node's services
  - Replication Monitor: Do we have too few or too many replicas?
  - Cluster Balancer: Are blocks well distributed across the cluster?

- Re-Replication is typically the result of a Data Node failure
  - Name Node detects out-of-contact Data Nodes via missing heartbeats
  - Marks them as 'dead'
  - Triggers Re-Replication

- Common setup is 3 replicas
  - Default Policy:
    - Replica 1: Random Data Node in the same rack as the Name Node
    - Replica 2: Random Data Node in a different (remote) rack
    - Replica 3: Different node in the same remote rack
  - If no racks, randomly allocate across nodes in the same rack
  - If the writer is on the same machine as a Data Node, replica 1 goes to that data node

University of Glasgow

Storage Types and Storage Policies

- Recently Hadoop introduced the idea of **Storage Type and Policies**
  - Aims to allow for intelligent placement of blocks based on their usage
  - Distinguished between Archive, Disk, SSD and RAM_Disk storage types

- Storage Policies are defined on Directories or Files and filter the available Data Nodes to those with particular types of storage
  - Examples: Hot, Cold, Warm, All_SSD
  - Specifies:
    - A primary block placement strategy for a given number of replicas
      - E.g. **Warm**: DISK: 1, ARCHIVE: *n*-1
    - A fallback storage type for initial creation (e.g. ARCHIVE, DISK)
    - A fallback storage type for replication (e.g. ARCHIVE, DISK)

https://hadoop.apache.org/docs/r3.3.4/hadoop-project-dist/hadoop-hdfs/ArchivalStorage.html

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers
@UofGlasgow